



Module 03 : Identity and Access Management

Root Account (Root User)

The "root user" refers to the account created when signing into AWS using the email and password associated with the account. This user has unrestricted access to the AWS account and can perform critical actions such as:

- Changing the support plan
- Modifying payment methods
- Closing the AWS account
- Transferring ownership of the account

The root user functions is similar to an admin on a computer, with the ability to install, modify, or delete any aspect of the system.

Given the level of access, it is crucial to secure the root user. This can be done by enabling multi-factor authentication (MFA) or two-factor authentication (2FA) to add an extra layer of security.

Multi Factor Authentication for Users

Enabling MFA for AWS Users:

1. After logging into your AWS account, search for the IAM service.
2. Under **Security Recommendations**, choose to add MFA for the root user.
3. Alternatively, in the top-right corner of the dashboard, click on **Security Credentials**.
4. Under **Multi-factor authentication (MFA)**, select **Assign MFA device**.

Types of MFA Devices:

1. **Authenticator App**: Applications like Google Authenticator, Microsoft Authenticator, etc., generate one-time codes.
2. **Security Key**: A physical device (similar to a USB drive) that must be connected to your computer or laptop.
3. **TOTP (Time-based One-Time Password Token)**: A hardware device (resembling a USB drive) that continuously displays a six-digit code, which must be entered during login. This functions like an RSA token.

Note: The **Security Key** is considered the most secure option, as it physically needs to be present. In contrast, apps or TOTP devices can expose their codes if left unattended or forgotten in places like home.

IAM User: Identity and Access Management.

IAM (Identity and Access Management) allows us to create users and groups, and manage their permissions to access AWS resources.

Example:

In a large project, multiple resources may be involved, and IAM helps us assign specific permissions based on roles:

1. **AWS Engineer:**
 - For handling **EC2/server issues**, we create a user with their own username and password, but restrict their access to only specific resources. For example, they can be given **EC2 full access**, but any attempt to perform actions outside of this scope would be denied.

2. Database Administrator:

- For resolving **database issues**, a separate user with individual credentials is created, and they are granted **RDS full access**.

3. Monitoring:

- To **monitor AWS resources**, a user is created with **read-only access**, allowing them to view resources but not make any changes.

Key Concepts:

- **Least Privilege Mechanism:** This principle ensures that users are granted the minimum permissions necessary to perform their job functions. IAM allows us to enforce this by restricting user access to specific operations and resources based on their roles.

 **Important for Certification:** Understanding and implementing least privilege is crucial for security best practices.

 **Note:** In real-world organizations, users rarely operate as root users. Instead, they work as IAM users with specific permissions tailored to their role.



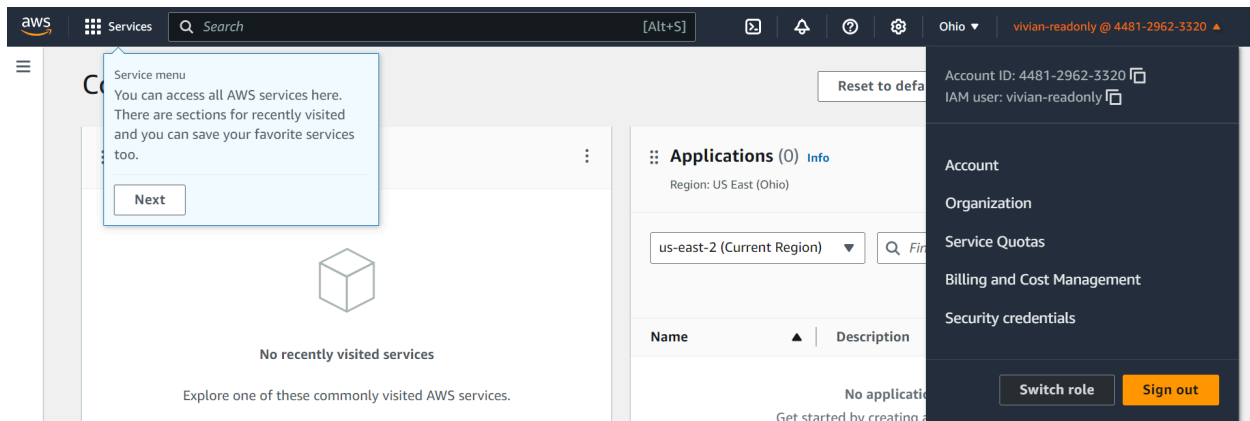
An AWS account can only have one root user; multiple root users are NOT allowed for a single account. It's not recommended to share root credentials with everyone. By creating an IAM multiple users and manage their access. This is the main advantage of IAM.

IAM is completely FREE always; no limitation.

How to Identify if You Are Logged in as a Root User or IAM User

- After logging in, click on the account information in the top-right corner of the console.
- If only the 12-digit account ID is displayed, you are logged in as the **root user**.
- If the display shows "username@account-id," you are logged in as an **IAM user**.

This distinction helps you quickly identify the type of account access you are using.



Creating read-only IAM user

1. Log in to the AWS Management Console:

- Sign in using your root account or an IAM account with administrative privileges.

2. Navigate to the IAM Dashboard:

- From the AWS Management Console, type "IAM" in the search bar and select **IAM**

3. Create a New User:

- In the IAM dashboard, click on **Users** in the left-hand menu.
- Click the **Create User** button.

4. Set User Details:

- Enter a **Username** for the new user.



When creating users in AWS, there are two primary types of access:

1. **Management Console Access** – Allows the user to log in via a web browser.
2. **Programmatic Access** – Enables access through the AWS CLI, SDKs, or APIs (covered later in the course).

For now, we're focusing on **Management Console Access**.

Next, You will be prompted with: **“Are you providing console access to a person?”**

You will see two options:

- **Specify a user in IAM Identity Center** – Identity center topic is covered later in the course.
- **I want to create an IAM user** – Choose this option for now and proceed to set credentials for the new IAM user.

4.1 Enable Console Access

- Check the box: **“Provide user access to the AWS Management Console – optional”** to allow web-based login.
- Select the user type: **“I want to create an IAM user”** – Choose this option for now and proceed to set credentials for the new IAM user.
- **Console Password:** Set the user's console password, either by choosing an auto-generated password or specifying your own password.

Search [Alt+S] Global prod

ers > Create user

Step 3
○ Review and create
Step 4
○ Retrieve password

Create user

User name

iam-admin-billing-test

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Are you providing console access to a person?

User type

☐ **Specify a user in Identity Center - Recommended**
We recommend that you use Identity Center to provide console access to a person. With Identity Center, you can centrally manage user access to their AWS accounts and cloud applications.

☒ **I want to create an IAM user**
We recommend that you create IAM users only if you need to enable programmatic access through access keys, service-specific credentials for AWS CodeCommit or Amazon Keyspaces, or a backup credential for emergency account access.

Console password

☒ **Autogenerated password**
You can view the password after you create the user.

☐ **Custom password**
Enter a custom password for the user.

- Must be at least 8 characters long
- Must include at least one uppercase letter (A-Z)
- Must include at least one lowercase letter (a-z)
- Must include at least one number (0-9)
- Must include at least one non-alphanumeric character (! @ # \$ % ^ & * () _ + = [] { } |)

☐ Show password

☒ **Users must create a new password at next sign-in - Recommended**

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.
[Learn more](#)

Cancel Next

5. Set Permissions:

- On the **Permissions** page, choose **Attach policies directly**.
- Search for and select the policy named **"ReadOnlyAccess"**. This grants the user read-only access to all AWS services.
- Skip this as shown in the lecture video - **"Set permissions boundary - optional"**



When assigning permissions to a new IAM user, you have 3 options:

1. Add User to a Group

Create a new group (or use an existing one), assign policies to that group, and then add the user to it.



Recommended for managing permissions at scale.

2. Copy Permissions from an Existing User

Duplicate the permission set from another IAM user.



Useful for quickly onboarding users with similar roles.

3. Attach Policies Directly

Assign one or more IAM policies directly to the user.



Not ideal for long-term management, especially in larger environments.

6. Review and Create:

- Review the user's details on the **Review** page to ensure everything is correct.
- Click **Create User** to finalize the process.

7. Provide Access Information:

- After creating the user, AWS will display access credentials (if console access was enabled) and login details for console access.
- Download or copy this information and share it with the user securely.

The new IAM user now has read-only access across your AWS account.

Creating a storage administrator user group with full access to S3 only

1. Log in to the AWS Management Console

- Sign in with your root or an IAM user with administrative privileges.

2. Navigate to the IAM Dashboard

- In the AWS Management Console, type **IAM** in the search bar and select **IAM**.

3. Create a New Group for S3 Administrators

- In the IAM dashboard, click on **User Groups** in the left-hand menu.
- Click **Create Group**.

Steps for Creating the Group:

- **Group Name:** Enter a name like `S3-Admins`.
- **Attach Policies:**
 - In the policy search bar, type `AmazonS3FullAccess`.
 - Select the policy **AmazonS3FullAccess** to give the group full access to all S3 buckets and objects.
- Click **Create Group**.

4. Create the IAM User (s3Admin-user)

- On the IAM dashboard, click **Users** in the left-hand menu.
- Click **Add User**.

Steps for Adding the User:

- **User Name:** Enter the username `s3Admin-user`.
- **Access Type:**
 - If console access is needed, select **AWS Management Console access** and choose either an autogenerated or custom password.
 - If programmatic access (API, CLI, or SDK access) is required, select **Programmatic access**.
- Click **Next: Permissions**.

5. Assign the User to the S3-Admins Group

- On the **Permissions** page, choose **Add user to group**.
- Select the group you just created, **S3-Admins**.
- Click **Next**.

6. Review and Create the User

- Review the user details to ensure everything is correct.
- Click **Create User** to finish.

7. Provide Access Information

- If you enabled programmatic access, download the **access keys** (Access Key ID and Secret Access Key).
- Provide the user with their console login details and/or access keys.

Creating IAM user with administrative access

The **root user** has complete, unrestricted access to all resources and settings in an AWS account. In contrast, an **IAM user with administrative access** has nearly the same level of control but with certain limitations. While the IAM administrator can manage services and resources across AWS, they cannot perform sensitive account management tasks such as:

- Closing the account
- Changing the support plan
- Modifying the payment methods

These specific actions are reserved for the root user only.

1. Log in to the AWS Management Console:

- Sign in using your root account or an IAM account with administrative privileges.

2. Navigate to the IAM Dashboard:

- From the AWS Management Console, type "IAM" in the search bar and select **IAM**

3. Create a New User:

- In the IAM dashboard, click on **Users** in the left-hand menu.
- Click the **Create User** button.

4. Set User Details:

- Enter a **Username** for the new user.
- Check the box “Provide user access to the AWS Management Console - *optional*” for **AWS Management Console access** (for console access).
- If console access is selected, set the user's console password, either by choosing an auto-generated password or specifying your own password.

5. Steps for Adding the User:

- **User Name:** Enter the desired username (e.g., `admin-user`).
- **Access Type:**
 - For management console access, check **AWS Management Console access** and choose either an autogenerated or custom password.
 - For programmatic access, check **Programmatic access** to allow API, CLI, or SDK access.
- Click **Next: Permissions**.

6. Assign Administrator Access

- On the **Permissions** page, choose **Attach policies directly**.
- In the policy search bar, type `AdministratorAccess`.
- Select the policy **AdministratorAccess**, which grants full permissions across all AWS services.
- Click **Next**.

7. Review and Create the User

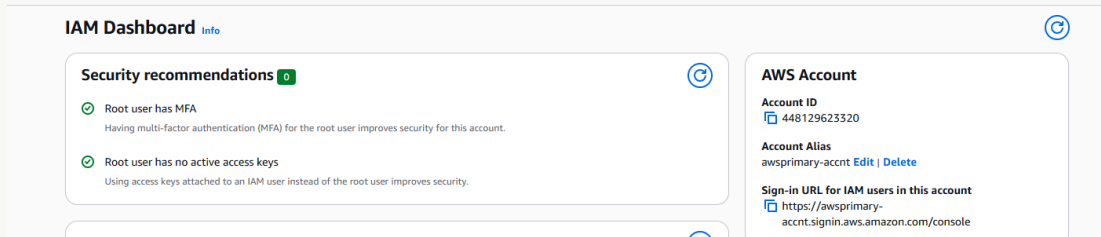
- On the **Review** page, verify that the correct username and policies are assigned.
- Click **Create User** to complete the process.

8. Provide Access Information

- If you enabled programmatic access, download the **Access Key ID** and **Secret Access Key**.
- For console access, note the login URL and credentials for the user.



- All IAM users share a common login URL. For example:
<https://xxxxxxxxxxxxxxxxx.signin.aws.amazon.com/console>



- It's important to note that **renaming an IAM username is NOT supported**.

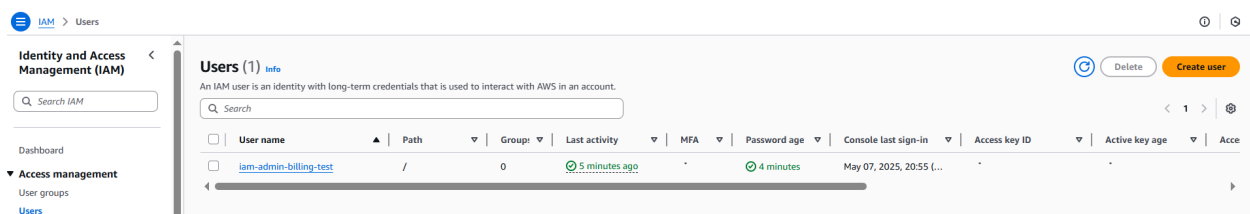
Assignments

Task 1: Create an IAM user, Add "AdministratorAccess" policy. Login and try to access "Billing information". It will deny.

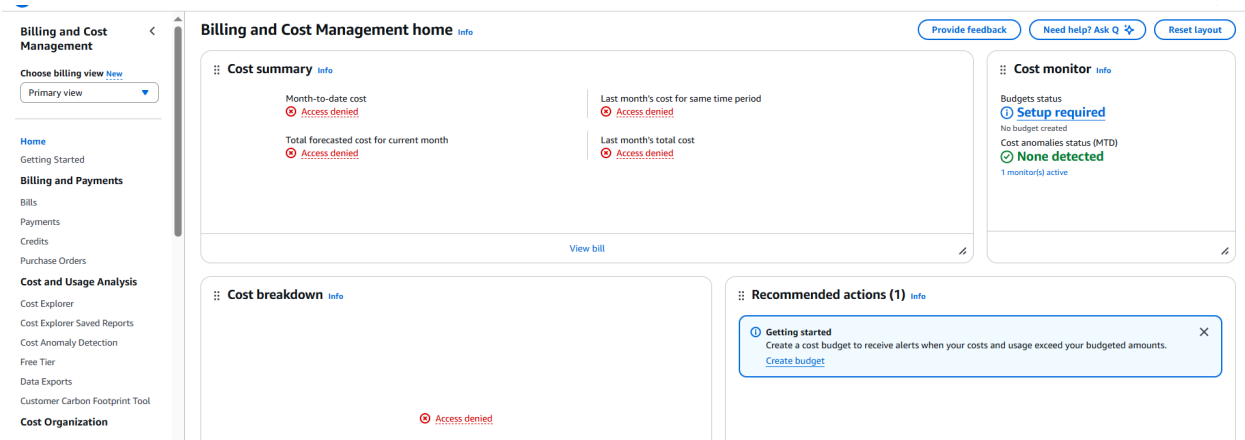
--> provide Billing access to the above IAM user.

IAM User Creation:

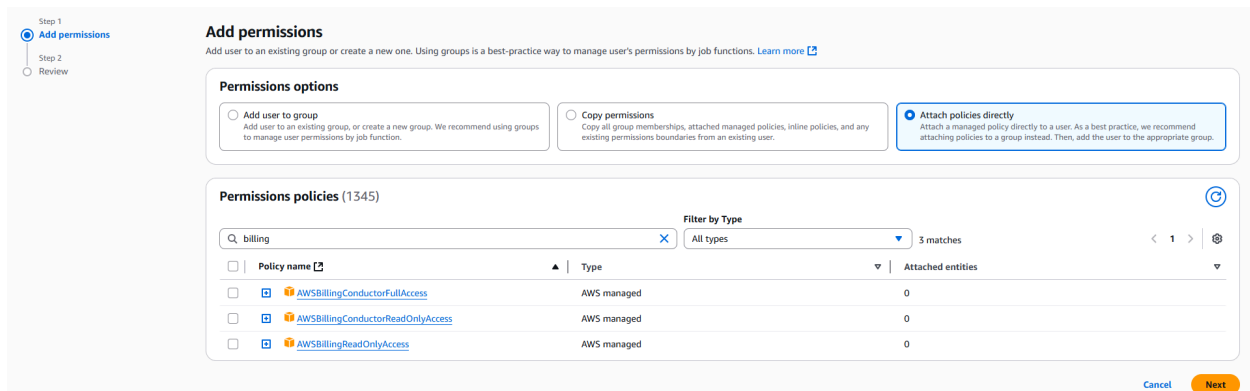
User name	Password	Console sign-in URL
iam-admin-billing-test	CdE98W**	https://awsprimary-acct.signin.aws.amazon.com/console



Billing and Cost Management (Access Denied):

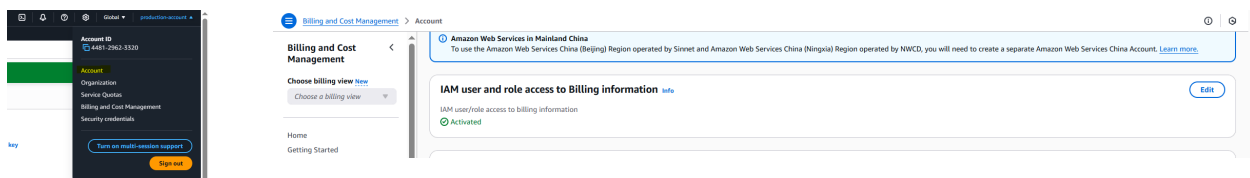


Provided Billing Access:



Billing and Cost Management (Access Enabled):

1. Log in as the **root user** (or a user with root-level access).
2. Go to **Account** → scroll down to “**IAM User and Role Access to Billing Information.**”

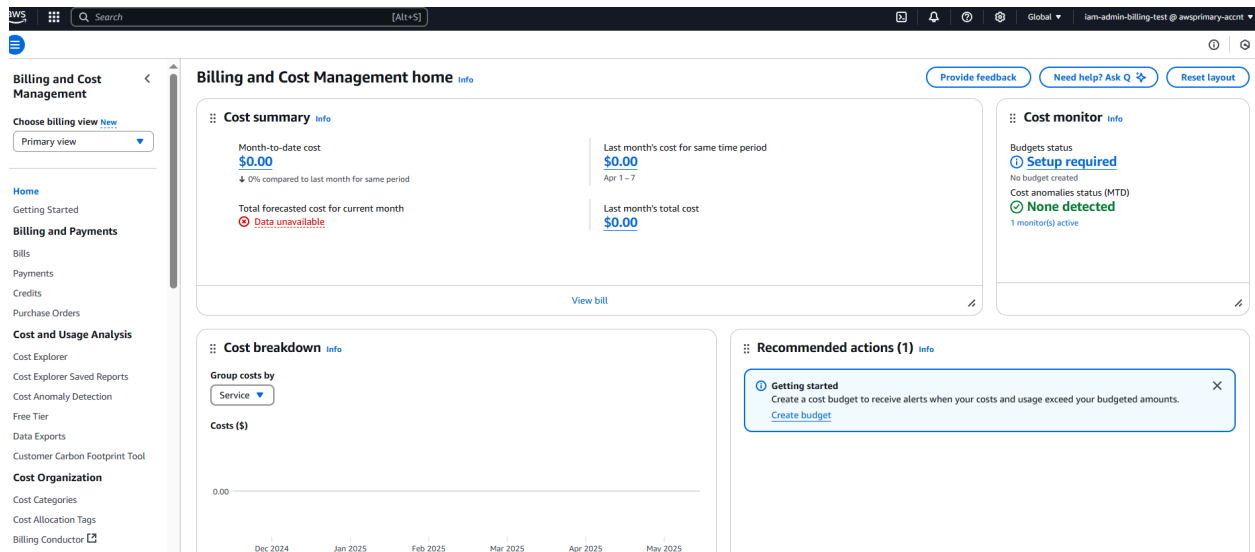


3. Click **Edit**, and check the box:

☒ "Activate IAM Access to Billing Information".

4. Click **Update** to save the setting.

Once this is enabled, IAM users with the appropriate permissions (e.g., **AdministratorAccess**) can access the billing dashboard.



Task 2 : create an IAM user, Add "S3FullAccess", login as this user and create a bucket. Now navigate to IAM service and try to create an IAM user and verify.!

Step 1: IAM User Creation

User name	Password	Console sign-in URL
s3-restricted-iamtest	gE[u_Q1**	https://awsprimary-accnt.signin.aws.amazon.com/console

IAM > Users > Create user

Step 1
Specify user details
Step 2
Set permissions
Step 3
Review and create
Step 4
Retrieve password

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name

s5-restricted-iamtest

Console password type

Autogenerated

Require password reset

Yes

Permissions summary

Name

AmazonS3FullAccess

Type

AWS managed

Used as

Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel Previous Create user

Step 2: Creating an S3 bucket

Amazon S3 > Buckets

Successfully created bucket "iam-s3-bucket-demo"

To upload files and folders, or to configure additional bucket settings, choose [View details](#).

Account snapshot - updated every 24 hours

Storage lens provides visibility into storage usage and activity trends. Metrics don't include directory buckets. [Learn more](#)

General purpose buckets | Directory buckets

General purpose buckets (1)

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	IAM Access Analyzer	Creation date
iam-s3-bucket-demo	US East (Ohio) us-east-2	View analyzer for us-east-2	May 7, 2025, 21:50:16 (UTC-05:00)

Step 3: Create IAM User

IAM > Users

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

User groups

Roles

Policies

Identity providers

Account settings

Root access management

Access reports

Access Analyzer

External access

Unused access

Users (0)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

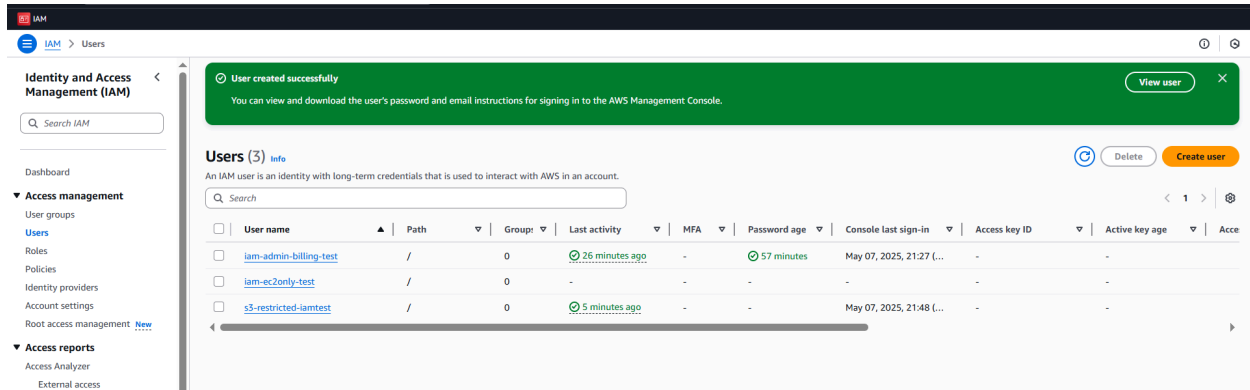
Search

User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Access key ID	Active key age	Access key last used
Access denied You don't have permission to <code>iam:ListUsers</code>. To request access, copy the following text and send it to your AWS administrator. Learn more about troubleshooting access denied errors. <pre> User: arn:aws:iam:448129623320:user/s3-restricted-iamtest Action: iam:ListUsers On resource(s): arn:aws:iam:448129623320:user/ Context: no identity-based policy allows the action </pre>									

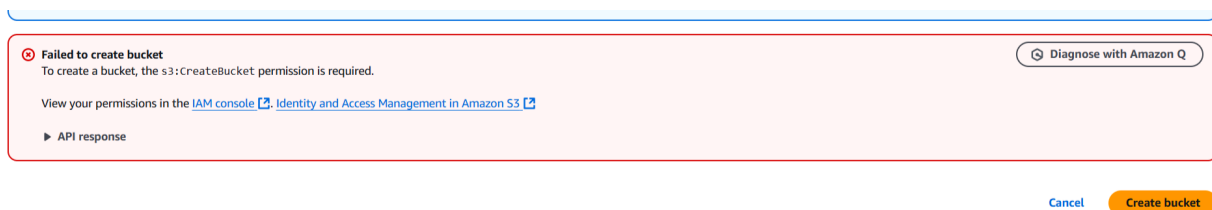
Task 3 : create an IAM user, Add "EC2FullAccess", login as this user and try to create a bucket.

Step 1: Create IAM User

User name	Password	Console sign-in URL
iam-ec2only-test	ndX8@zE_*	https://awsprimary-accnt.signin.aws.amazon.com/console



Step 2: Create S3 Bucket



Task 4: Setup password standard and account alias for your AWS account

IAM → Account Settings under Access Management → Edit the "Password policy"

- Navigate to **IAM** in the AWS Console.
- Under **Access Management**, go to **Account Settings**.
- Click **Edit** next to **Password policy** to configure your organization's password standards (e.g., minimum length, complexity, expiration rules).
- You can also set a custom **Account Alias** here, which replaces the default AWS account ID in the login URL with a more user-friendly name (e.g., `yourcompany.signin.aws.amazon.com/console`).

IAM
Account Settings

Identity and Access Management (IAM)

Dashboard
Access management
Users groups
Users
Roles
Policies
Identity providers
Account settings
Root access management
Access reports
Access Analyzer
External access
Unused access
Analyzer settings
Credential report
Organization activity
Service control policies
Resource control policies

Account settings

Password policy

Configure the password requirements for the IAM users.

This AWS account uses the following custom password policy:

Password minimum length
8 characters

Password strength

- Require at least one uppercase letter from the Latin alphabet (A-Z)
- Require at least one lowercase letter from the Latin alphabet (a-z)
- Require at least one number
- Require at least one non-alphanumeric character

Other requirements

- Password expires in 60 day(s)
- Allow users to change their own password
- Prevent password reuse from the past 2 changes

Security Token Service (STS)

STS is used to create and provide trusted users with temporary security credentials that can control access to your AWS resources.

Session Tokens from the STS endpoints

AWS recommends using regional STS endpoints to reduce latency. Session tokens from regional STS endpoints are valid in all AWS Regions. If you use regional STS endpoints, no action is required. Session tokens from the global STS endpoint (https://sts.amazonaws.com) are valid only in AWS Regions that are enabled by default. If you intend to enable a new Region for your account, you can use session tokens from regional STS endpoints or activate the global STS endpoint to issue session tokens that are valid in all AWS Regions.

Global endpoint
Valid only in AWS Regions enabled by default | [Change](#)

Regional endpoints
Valid in all AWS Regions

Endpoints (10) ...

IAM
Account Settings
Edit password policy

Edit password policy

Password policy

☐ IAM default
Apply default password requirements.
☒ Custom
Apply customized password requirements.

Password minimum length.
Enforce a minimum length of characters.

8 characters
Needs to be between 6 and 128.

Password strength

- ☒ Require at least one uppercase letter from the Latin alphabet (A-Z)
- ☒ Require at least one lowercase letter from the Latin alphabet (a-z)
- ☒ Require at least one number
- ☒ Require at least one non-alphanumeric character (!@#\$%^&*()_+-=[\|'`)

Other requirements

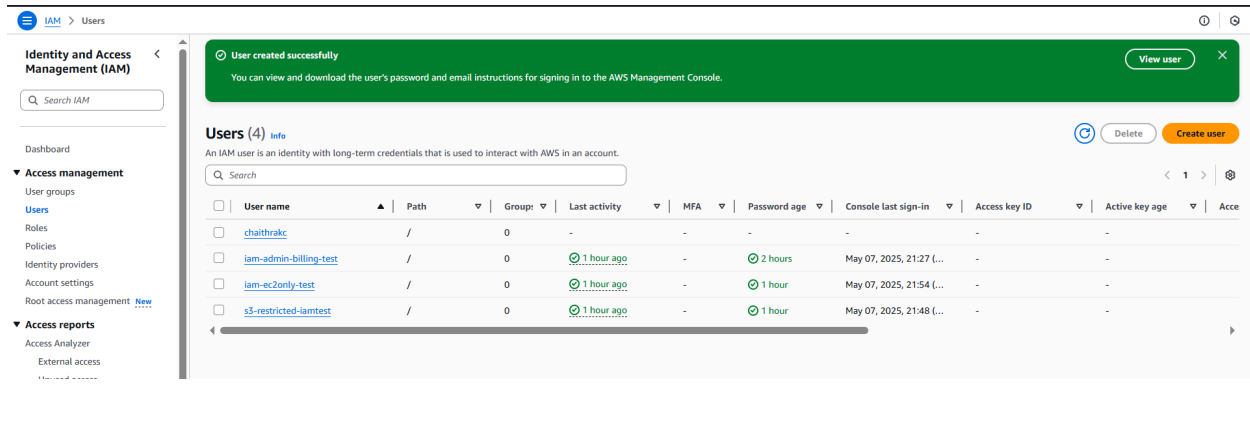
- ☐ Turn on password expiration
- ☐ Password expiration requires administrator reset
- ☒ Allow users to change their own password
- ☒ Prevent password reuse

Remember 2 password(s)
Needs to be between 1 and 24.

Cancel
Save changes

IAM User Creation

User name	Password	Console sign-in URL
chaithrakc	xvV!0{Q}*	https://awsprimary-accnt.signin.aws.amazon.com/console



Policy

A policy is a document that defines a **set of permissions** for AWS services, controlling which actions can be performed and on which resources. Policies are written in JSON format. Policy provide permissions on AWS resources for users/groups/roles.

When we are creating IAM user, we are associating policy. Policy is nothing but set of permissions on our AWS account. Based on the common use cases, AWS is automatically given the policies. For example: S3ReadOnly, S3FullAccess, EC2FullAccess.

3 types of policies:

- AWS Managed Policy : Policy provided by AWS, Based on Service.
- AWS managed - Job function policy : Policy provided by AWS. It is a combination of multiple Services. AWS provided this based on some standard jobs available in market.
- Customer Managed policy : Based on customer requirement, we can create and manage.

Creating Customer Managed Policies

Sometimes the given AWS policies won't satisfy our requirement. For example, a policy should allow two or three services. AWS gives a policy for individual services most of the time. If it combines multiple services, its called Job Function.

Creating a custom policy that allows both S3 and EC2 full access:

In the IAM (Identity and Access Management), go to policies, click on "create policy".

1. Specify permissions

- a. We have JSON and Visual editor. Most of the time, we use Visual editor. So choose visual editor
 - b. Select a service - S3
 - c. Effect - Allow/ Deny : choose Allow
 - d. Actions allowed - select All S3 actions
2. Click on “add more permissions”
- a. Select a service - EC2
 - b. Effect - select Allow operation
 - c. Actions - All EC2 actions
 - d. Resources - “All”

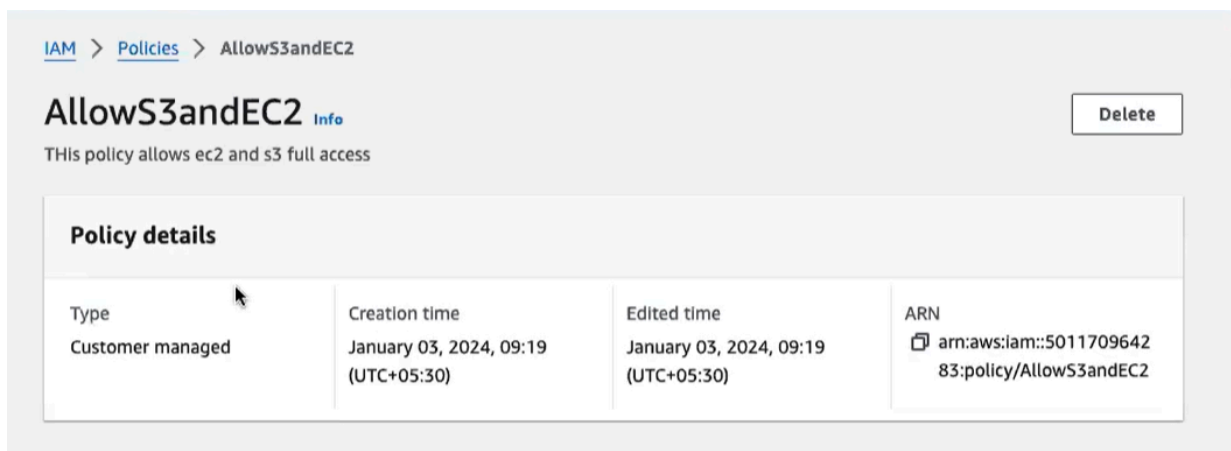
Click Next

3. Policy Details

- a. policy name - “AllowS3andEC2”
- b. Description - This policy allows EC2 and S3 full access.

Click on “Create Policy”.

Observe the policy type in the below screenshot, the policy type is “Customer managed”.



Example of AWS managed policy:

IAM > Policies > AccessAnalyzerServiceRolePolicy

AccessAnalyzerServiceRolePolicy Info

Allow Access Analyzer to analyze resource metadata

Policy details			
Type	Creation time	Edited time	ARN
AWS managed	December 02, 2019, 22:43 (UTC+05:30)	November 29, 2023, 00:30 (UTC+05:30)	 arn:aws:iam::aws:policy/awsservice-role/AccessAnalyzerServiceRolePolicy



AWS managed policies **CANNOT** be deleted or modified.

Customer managed policies **CAN BE** deleted or modified anytime. Most of the times in real environment, we go with customer managed policies.

Implicit Allow Mechanism

Access is restricted to only the allocated services. Any attempt to access unassigned services will be denied automatically.

AWS IAM uses an **implicit deny** by default:

- **Users have no permissions unless explicitly granted.**
- Any attempt to access a service or resource that is not explicitly allowed will be **automatically denied**.
- This ensures secure, least-privilege access by default.

Explicit Deny Mechanism

Requirement:

You have an

IAM user that requires access to **all AWS services, except S3**. The goal is to **restrict S3 access** while still granting broad permissions. Can this be achieved using **AWS Managed Policies**?

Solution Overview:

AWS managed policies support the **implicit allow** model. You can attach multiple AWS managed policies that grant access to all services **excluding S3**. However, this approach has limitations:

- Maximum **20 managed policies per user**
- Maximum **50 managed policies per group**

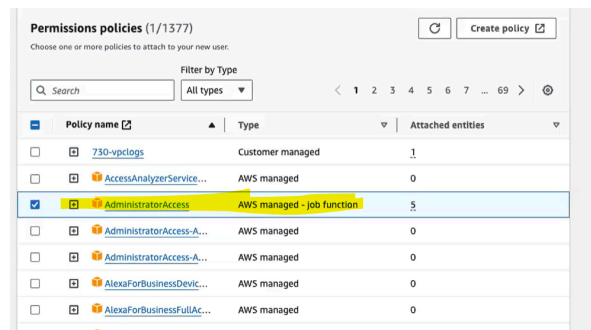
Due to these limits, it's impractical to attach all AWS service policies **except S3** using AWS managed policies.

Recommended Approach

To meet the requirement, follow a **two-part strategy** using **custom (customer managed) policies**:

Part 1: Grant Broad Access

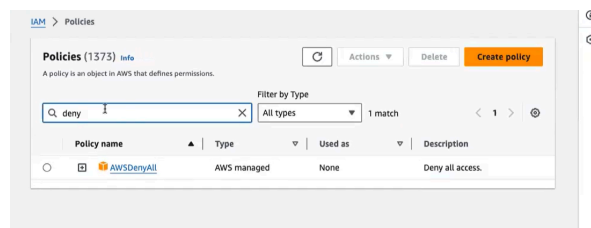
- Attach the **AdministratorAccess** managed policy to the user.
- This grants full access to all AWS services.



Second part of the requirement is to deny. AWS managed policy has only one deny policy - “AWSDenyAll” which does not meet our requirement to deny only S3 service. So, in this scenario we need to create our own policy.

Part 2: Explicitly Deny S3 Access

- AWS only provides one managed deny policy: **AWSDenyAll**, which **denies everything**. This doesn't meet our specific need to **only deny S3**.
- **Create a custom policy** to explicitly **deny S3 actions**.

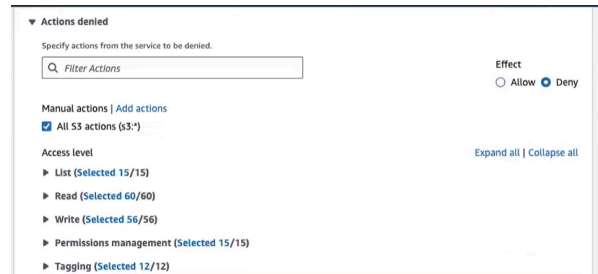


when creating the new policy, select deny mechanism.

Steps to Create a Deny Policy

1. Go to IAM → Policies → Create Policy
2. Choose the **JSON editor**.
3. Define a policy like this:

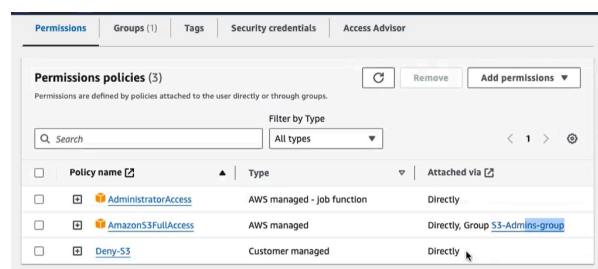
```
json
CopyEdit
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "s3:*",
      "Resource": "*"
    }
  ]
}
```



4. Name the policy something like **DenyS3Access** and save it.

Attach the Deny Policy

- Go to IAM → Users → [Your User] → **Permissions**
- Click “Add Permissions” → “Attach policies directly”
- Select and attach your custom **DenyS3Access** policy



Attach the Deny Policy (alternate way)

- Go to the IAM console.

- Navigate to the "**Policies**" section.
- Locate and select the policy you want to attach (e.g., `DenyS3Access`).
- In the policy details page, click on the "**Permissions**" tab.
- Under "**Entities attached**", click the "**Attach**" button.
- Choose the appropriate **users, groups, or roles** to attach the policy to.
- Click "**Attach Policy**" to apply it.

The screenshot shows the AWS IAM console interface. At the top, a green notification bar states "Policy Deny-S3 created." with a "View policy" button. Below this, a table lists policy details: "Customer managed", "January 03, 2024, 09:31 (UTC+05:30)", "January 03, 2024, 09:31 (UTC+05:30)", and "arn:aws:iam::501170964283:policy/Deny-S3". The "Entities attached" tab is selected, showing "Attached as a permissions policy (0)". It includes a search bar, a "Filter by Entity type" dropdown set to "All types", and a table with columns "Entity name" and "Entity type". The table is currently empty, with a message "Policy not attached to any entity." and an "Attach" button.



Key Principle: Deny Overrides Allow

Even if the user has multiple policies (e.g., `AdministratorAccess`, `AmazonS3FullAccess`, `S3-admins-group` policies), **any explicit deny takes precedence**.

If a user has 100 policies that allow an action, and just 1 that denies it, the deny wins.

This is the **Explicit Deny Mechanism** in AWS IAM.

Policy Generator

When creating or evaluating an IAM policy, several key components are used within the policy document. These are defined in the **JSON structure** of the policy.

Key Elements of a Policy Statement:

- **Statement** : A container for individual permission rules.
- **Sid** (Statement ID): An optional identifier for the statement.
- **Effect** : Determines whether the policy **allows** or **denies** the specified actions. Values:
 - **"Allow"**
 - **"Deny"**
- **Action** : Specifies the actions the policy applies to.

Example:

```
"Action": "s3:*"
```

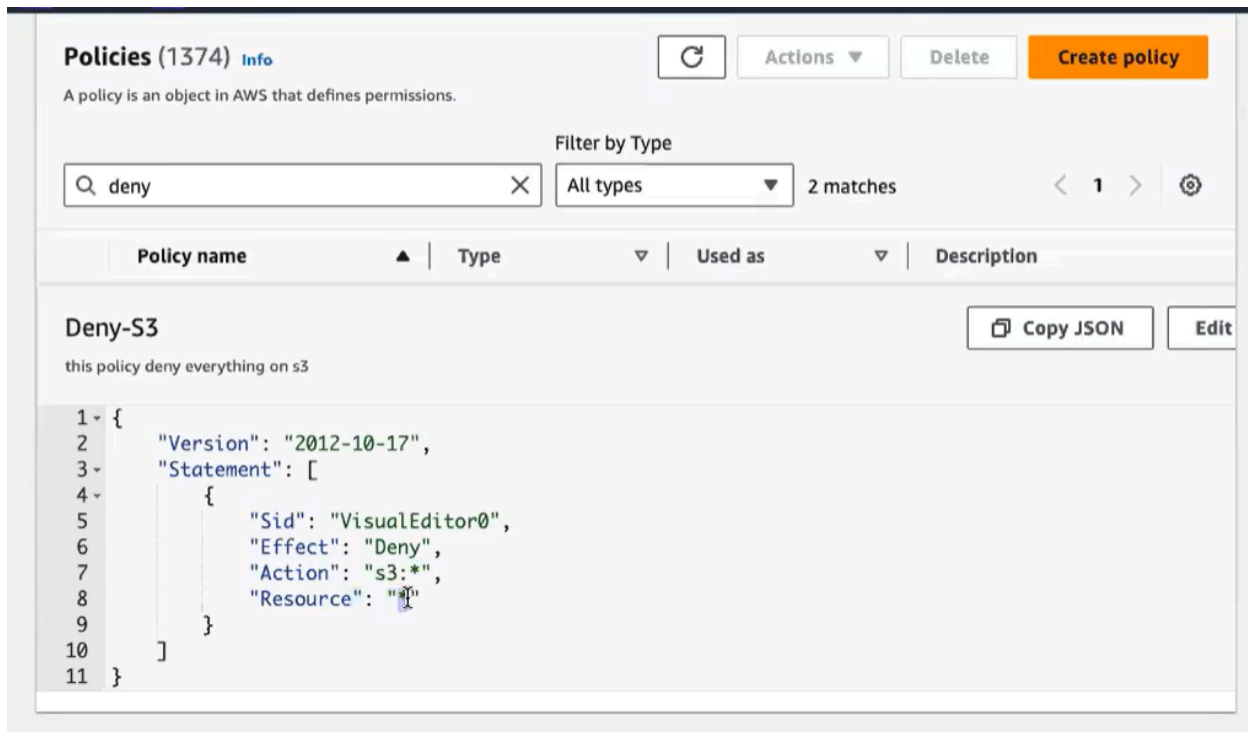
This grants or denies all actions for the **Amazon S3** service.

- **Resource** : Specifies the AWS resources the action applies to.

Example:

```
"Resource": "*"
```

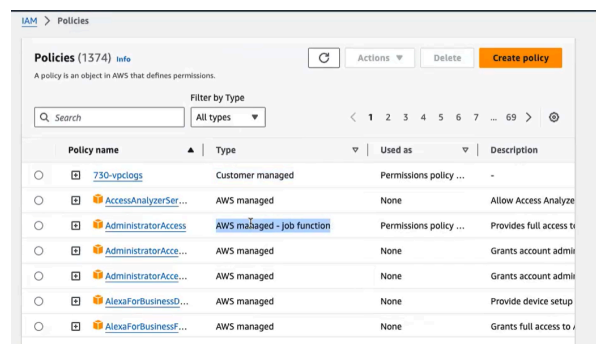
The asterisk (*****) is a **wildcard**, meaning **all resources**. The wildcard is commonly used in programming to indicate **"everything"** or **"all"**. In IAM policies, it's used to apply the rule broadly unless a more specific resource ARN is defined.



AWS Managed Job Function Policies

In addition to **AWS Managed Policies** and **Customer Managed Policies**, AWS also provides a special category called **Managed Job Function Policies**.

- **Job Function policies** are **predefined AWS managed policies** designed around **common industry job roles** such as:
 - Database Administrator (DBA)
 - Network Administrator
 - Data Scientist
 - Developer
 - Security Auditor
- These roles typically require a **mix of permissions** across multiple AWS services. For example:

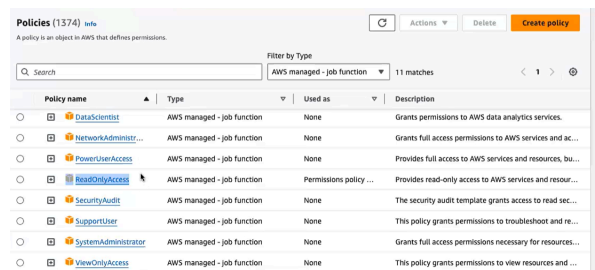
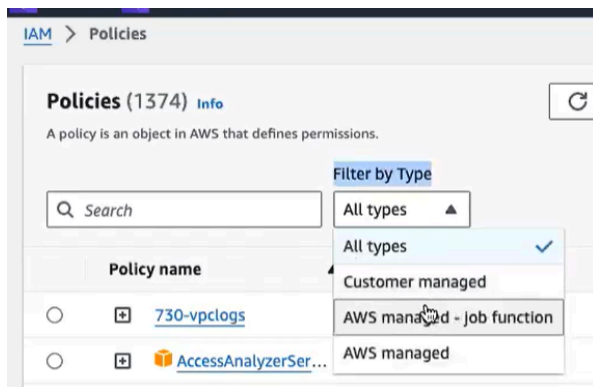


- A Data Scientist might need **read-only access** to some services (like IAM) but **full access** to others (like S3 or SageMaker).
- AWS has analyzed these typical access patterns and **consolidated them into standardized policies** for each job function.

How to Access Job Function Policies

You can find these policies in the IAM console:

- Go to **IAM** → **Policies**
- Use the **“Filter by Type”** option
- Select **“AWS managed – job function”** to view only job function policies





Customization Options

- You cannot modify AWS managed job function policies directly.
- However, you can copy the policy's **JSON definition**, create a **customer managed policy**, and then **customize it**.

Example Use Case:

If your organization wants to use the **DataScientist policy** but exclude access to **CloudWatch**, you can:

1. Copy the JSON from the AWS job function policy
2. Create a **new custom policy**
3. Remove permissions related to CloudWatch
4. Attach the customized policy to the relevant users or roles

Policy Versioning in AWS IAM

When you create a new IAM policy, it automatically starts with a **default version**. This is the initial version of the policy and is labeled as **v1**.

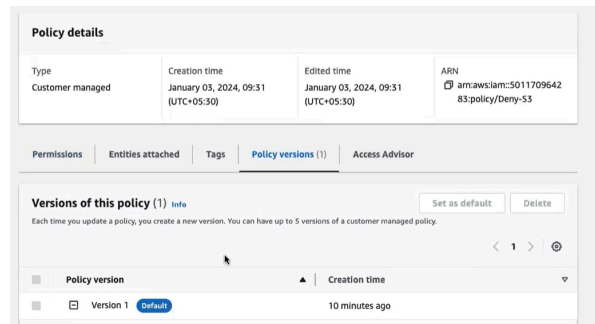
How Policy Versions Work

- AWS IAM supports up to **5 versions** of a policy at any given time.
- Every time you **edit and save changes** to a managed policy, a **new version** is created.
- Only **one version** can be set as the **default**—this is the version that IAM actually evaluates and enforces.
- Older versions are retained for **reference** or **rollback**, but only up to five versions can be stored. If you attempt to create a sixth version, **one of the existing versions must be deleted** first.

Where to View Policy Versions

To view or manage versions:

1. Go to the **IAM Console**
2. Navigate to **Policies**
3. Click on your policy
4. Go to the “**Policy versions**” tab to:
 - View different versions
 - Set a version as the default
 - Delete older versions if needed



Using Conditions in IAM Policies

In real-world scenarios, IAM policies often include **conditions** to enforce more granular access controls. Conditions allow you to **limit permissions based on specific attributes** such as IP address, time of day, MFA presence, and more.

Example Use Case: IP-Based Restriction

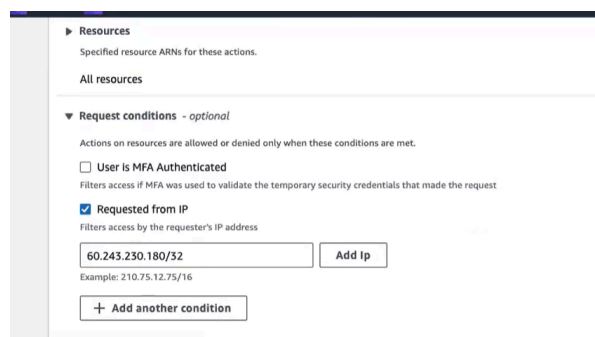
Let’s consider a scenario:

An **IAM user** has **admin access to Amazon S3**. By default, this user can access S3 from **anywhere in the world** as long as they have internet access. This means they can upload or download data even from unsecured networks.

However, your organization wants to **restrict this user’s access** so that they can only use S3 **from within the corporate office network**.

How to Apply the Condition

1. When creating or editing the IAM policy, add a **Condition** block to restrict access based on IP.
2. Use the **aws:SourceIp** condition key to specify the **IP address or subnet** of your corporate office.



3. You can determine your IP/subnet using services like:

<https://whatismyipaddress.com>

Example Condition in JSON:

```
"Condition": {  
  "IpAddress": {  
    "aws:SourceIp": "203.0.113.0/24"  
  }  
}
```

This condition ensures that the IAM user can **only access S3** when their request comes from the specified IP range (e.g., your corporate office).

Amazon Resource Name

A unique identifier for our resources.

An **Amazon Resource Name (ARN)** is a **unique identifier** assigned to every AWS resource. It allows AWS to distinctly identify resources across all regions and services.

General ARN Structure:

```
arn:partition:service:region:account-id:resource
```

However, depending on the resource type, some parts may be omitted (e.g., region or account ID).

Examples:



```
arn:aws:iam::501170964283:user/req-user  
arn:aws:iam::501170964283:group/S3-Admins-group  
arn:aws:s3:::avinash
```

Explanation of Components:

1. **arn:aws** The **partition** (usually **aws** for most public AWS resources).
2. **service** The AWS **service name** (e.g., **iam**, **s3**).
3. **region** (*optional*) May be blank for global services like IAM or S3.
4. **account-id** Your **AWS account ID** that uniquely identifies the owner of the resource.
Example: **501170964283**
5. **resource** The **specific resource path** within the service.

Examples:

- **user/req-user**
- **group/S3-Admins-group**
- **s3:::avinash**

6. Special Case: S3 Buckets

S3 bucket ARNs differ slightly:

- Example:

```
arn:aws:s3:::avinash
```

- **No account ID is included** in the ARN.
- This is because **S3 bucket names must be globally unique**, so the **bucket name itself ensures uniqueness**.



Every time you create a resource (like a user, group, or bucket), AWS automatically generates an ARN that you can use in policies and logs to refer to that resource uniquely.

IAM Policy Simulator

We can simulate IAM user permissions and get information about the performing activity (allow/deny).

The

IAM Policy Simulator is a tool provided by AWS that helps you **test and troubleshoot IAM permissions—without actually performing the actions**. Think of it like other simulators (e.g., Android emulators or flight simulators): it's a **safe environment** to test what actions a user **can or cannot perform**, based on the policies attached to them.

Why Use It?

In complex IAM setups, a user might have multiple policies (e.g., 4 policies attached), and you may encounter unexpected access issues. For instance:

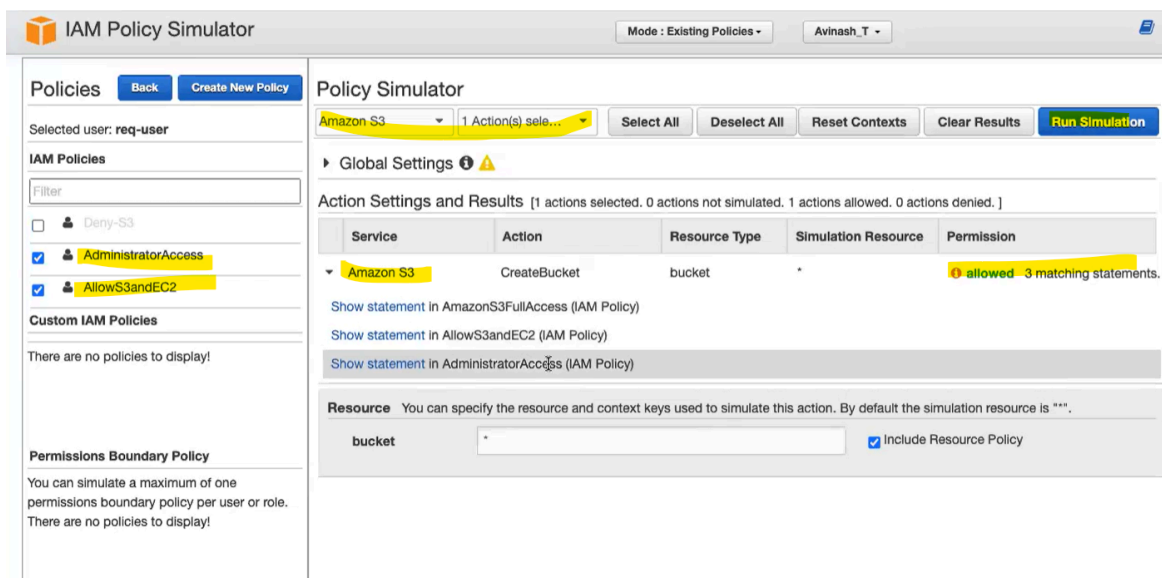
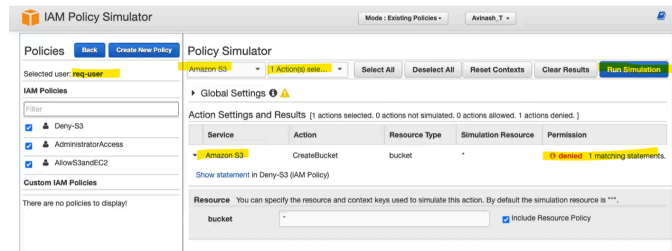
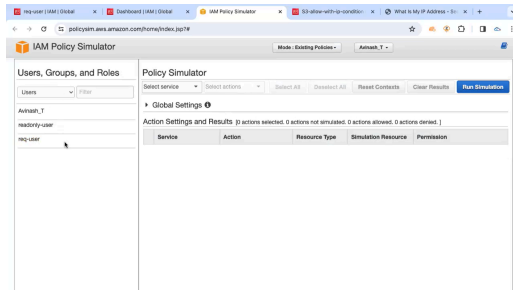
- The user **cannot create an S3 bucket**, but you're unsure **which policy is denying** the access.
- Instead of manually reading and analyzing each policy, the **Policy Simulator** helps you quickly pinpoint the **exact reason**—whether access is allowed or denied, and by which policy.

How to Use the IAM Policy Simulator

1. Go to the **AWS IAM Console**
2. Navigate to **Dashboard** → **Tools** → **Policy Simulator**
3. Select the **user, group, or role** you want to test
4. Choose the **service** and **action(s)** (e.g., `s3:CreateBucket`)
5. Run the simulation to see whether the action is **allowed** or **denied**, and **why**



The simulator doesn't make real API calls—it **only simulates permission evaluation**, making it a safe and effective tool for debugging access issues.



Task 1 : Create a policy to deny S3 service.

Follow the steps below to create a policy that blocks all actions on the S3 service:

1. Navigate to IAM Console

Go to **IAM** → **Policies** and click “**Create policy**”.

2. Select the Service

Choose “S3” as the target service.

3. Define Actions

- Under **Actions**, select “All S3 actions” (`s3:*`).

4. Set the Effect

- Choose “**Deny**” as the effect to explicitly block the specified actions.

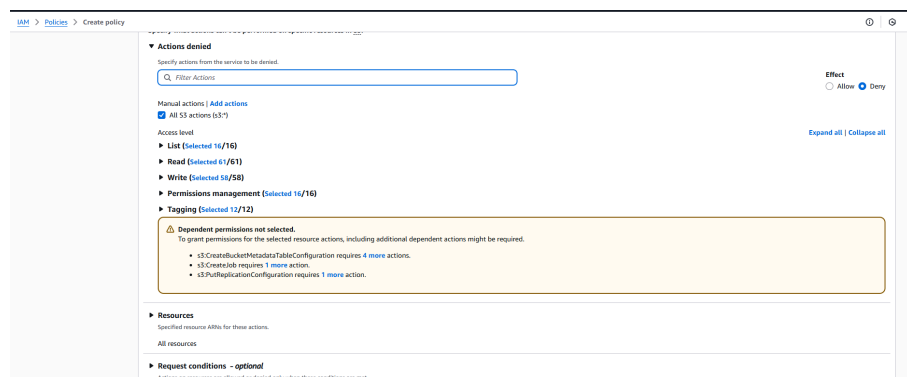
5. Specify Resources

- Set **Resources** to “**All**” to apply the denial to every S3 resource.

6. Review and Create

- Click **Next**, then enter:
 - **Policy Name:** `DenyS3Access`
 - **Description:** Explicitly denies all S3 service actions across all resources.
- Click “**Create Policy**” to finish.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditorDefault",
      "Effect": "Deny",
      "Action": "s3:*",
      "Resource": "*"
    }
  ]
}
```



☐	AWSQuickSetupSSMDeploymentsS3BucketRole...	AWS managed	None	This policy grants permissions for listi...
☐	AWSS3OnOutpostsServiceRolePolicy	AWS managed	None	Allow Amazon S3 on Outposts service ...
☐	DenyS3Access	Customer managed	None	This policy explicitly denies all actions ...
☐	IVSRecordToS3	AWS managed	None	Service Linked Role to perform S3 Put...

Task 2 : Create an IAM user, provide him "AdministratorAccess" and associate "task 1 policy" to this user, login and try to create a bucket.!!!

Console Sign-in URL: <https://awsprimary-accnt.signin.aws.amazon.com/console>

User name: testuser-deny-s3

Console password: UeVWaY8@

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name	testuser-deny-s3
Console password type	Autogenerated
Require password reset	No

Permissions summary

Name	Type	Used as
AdministratorAccess	AWS managed - job function	Permissions policy
DenyS3Access	Customer managed	Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Failed to create bucket

To create a bucket, the s3:CreateBucket permission is required.

View your permissions in the [IAM console](#) or [Identity and Access Management in Amazon S3](#)

API response

User: arn:aws:iam::448129623320:user/testuser-deny-s3 is not authorized to perform: s3:CreateBucket on resource: "arn:aws:s3:::testingdenys3access" with an explicit deny in an identity-based policy

Diagnose with Amazon Q

Cancel

Create bucket

Task 3 : Create a Policy to allow S3FullAccess, but this should work from only one/your Network.

** You have to add an IP address condition.

<https://whatismyipaddress.com/>

60.243.230.180/32

Step 1: Creating policy with condition

The screenshot shows the 'Request conditions' section in the AWS IAM console. It is titled 'Request conditions - optional' and explains that actions are allowed or denied based on these conditions. Two conditions are listed: 'User is MFA Authenticated' (unchecked) and 'Requested from IP' (checked). The 'Requested from IP' condition is expanded, showing the IP address '47.196.38.61/32' and a button to 'Add Ip'. Below this is a text input field labeled 'Enter IP address' with an example '210.75.12.75/16' and a '+ Add another condition' button.

Step 2: Testing whether we can remove a bucket from S3

The screenshot shows the 'Delete bucket' dialog in the AWS IAM console. It contains a warning box stating that deleting a bucket cannot be undone and that bucket names are unique. Below this is a section titled 'Delete bucket "test-bucket-deny-s3-access"?' with a text input field containing 'test-bucket-deny-s3-access'. At the bottom, there is a red error message: 'You don't have permission to delete bucket "test-bucket-deny-s3-access"'. The message explains that the user needs the 's3:DeleteBucket' permission in their IAM policy. There are 'Cancel' and 'Delete bucket' buttons at the bottom right.

Inline Policy

An **inline policy** is a policy that is **directly attached to a specific IAM entity** — either a **user**, **group**, or **role**. Unlike managed policies, inline policies are tightly bound to that single entity.

When to Use Inline Policies

Inline policies are typically used when:

- You're facing **policy count or size limitations** with managed policies and need an alternative way to grant permissions.
- You need a **one-to-one relationship** between a policy and an IAM entity.

Key Limitations

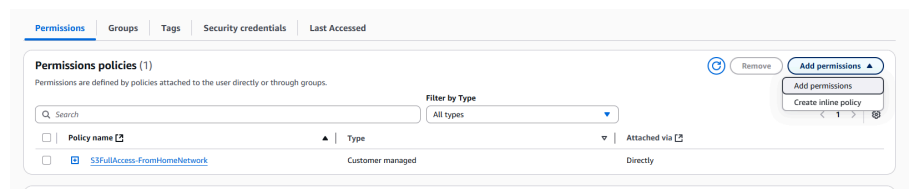
- **✗ Not Reusable:** Inline policies are attached to a single entity and **cannot be shared** across users, groups, or roles.
- **✗ Not Searchable or Listable:** They **do not appear in the IAM policy listing**, making them harder to audit and manage.
- **✗ Difficult to Track:** Since they don't have a standalone presence, you must manually inspect each user/group/role to find them.



Recommendation: If your use case requires reusability or centralized management, use customer managed policies instead of inline policies.

How to Create Inline Policies?

- **Go to the IAM Console**
- **Navigate to Users and select the user you want to attach the inline policy to.**
- **Click on the “Permissions” tab.**
- **Select “Add permissions” → “Create inline policy”.**
- **Follow the policy creation steps (same as creating a**



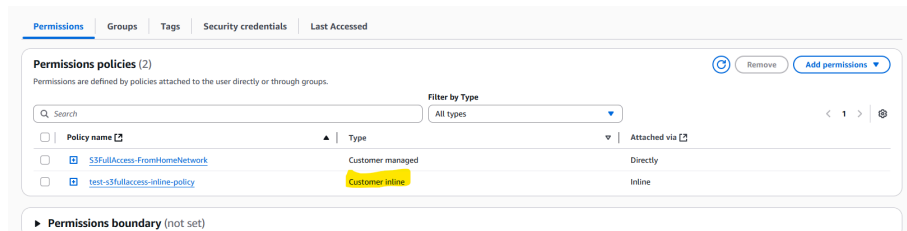
customer
managed
policy):

- Choose the
service (e.g.,
S3)
- Define
actions and
resources
- Optionally
set
conditions
- Review and
save the
policy



Note:

Inline policy creation interface is identical to that of customer managed policies—the key difference is that **inline policies are bound directly to a specific IAM entity** and cannot be reused.



Credential Report

A **Credential Report** is a downloadable CSV file generated from the IAM dashboard that provides a **comprehensive overview of IAM user credentials** in your AWS account.

It includes key security-related details such as:

- Last password usage

- Password expiration status
- Access key status and last used date
- Whether MFA is enabled
- Account creation dates

Why is the Credential Report Important?

Credential Reports are especially useful for:

- **Identifying Inactive Users**

When managing many users, it's inefficient to check login activity one by one. This report lets you quickly identify users who haven't logged in for months or years.

- **MFA Compliance**

You can spot which users have **not enabled MFA** and take action to enforce best security practices.

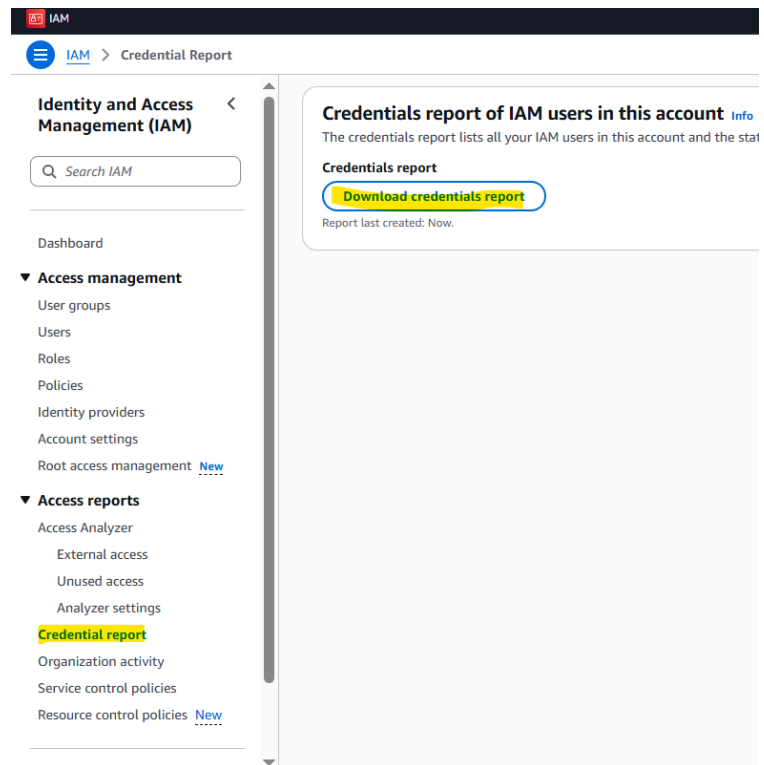
- **Periodic Audits and Cleanups**

Organizations often perform monthly or quarterly credential reviews to maintain hygiene and security. This report simplifies those audits.

How to Access the Credential Report

1. Go to IAM → Credential Report
2. Click “Download Report”

This will generate and download a CSV file with details for **all IAM users**.

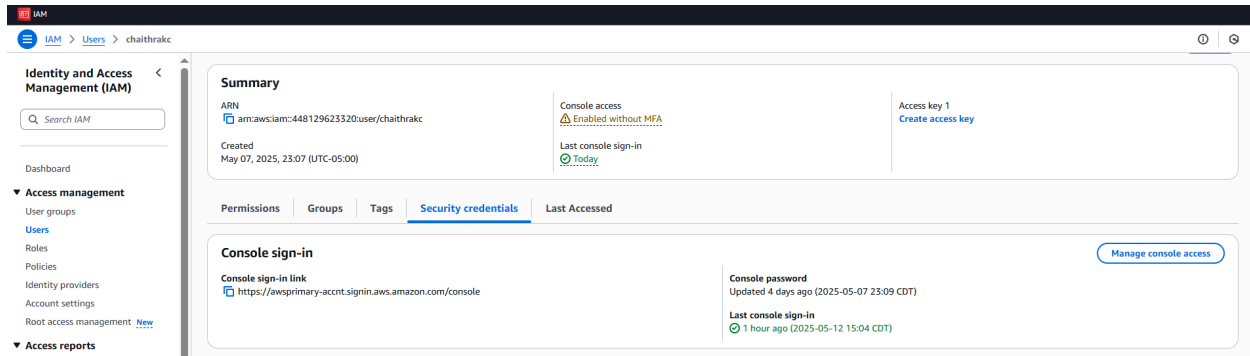


Alternative: View Individual User Credentials (Manual)

You can also view credentials for a specific user:

- Go to IAM → Users
- Select a user → go to the **Security Credentials** tab
- View details like **last console login** and **password last changed**

⚠ Note: This manual process is not scalable for large teams — use the credential report for organization-wide analysis.



Cloud trail

AWS CloudTrail is a service that **automatically logs all activities (API calls)** made across your AWS environment.

It provides **visibility into user and service actions** for auditing, security monitoring, and troubleshooting.

-  **Enabled by default**
-  **Stores logs for the last 90 days in Event History**

Real-World Use Case Example

As part of AWS operations, you might face scenarios like:

You granted S3 Full Access to 2–3 users, and later an S3 bucket was deleted.

You're asked: **"Who deleted the bucket?"**

 To investigate:

- Use **CloudTrail logs** to **trace the delete operation**.
- You can filter logs by **event name** (e.g., **DeleteBucket**), **username**, **date range**, and more.

◆ Using CloudTrail to Investigate Events

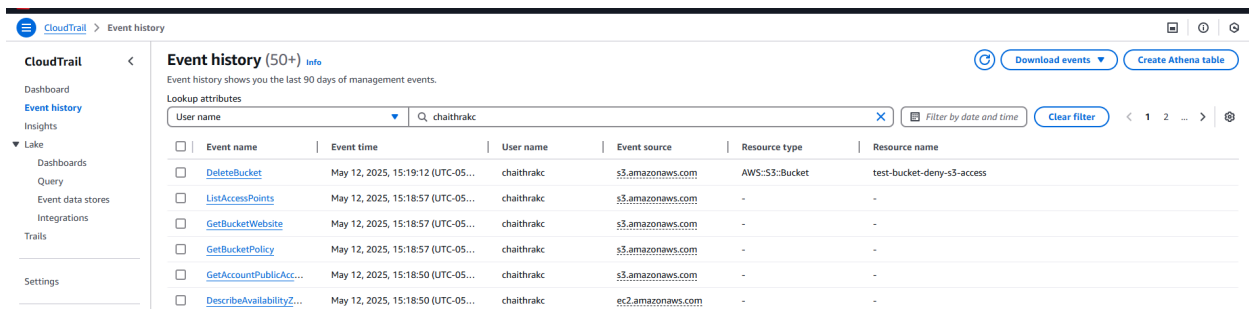
1. **Go to AWS Console** → Search for **CloudTrail**

2. In the **left navigation pane**, select **Event History**

3. Apply filters:

- Event Name: `DeleteBucket`
- Username: (optional)
- Time Range: Absolute or Relative

4. Review the event details or **download the logs** for further analysis



Event name	Event time	User name	Event source	Resource type	Resource name
DeleteBucket	May 12, 2025, 15:19:12 (UTC-05:00)	chaithrakc	s3.amazonaws.com	AWS::S3::Bucket	test-bucket-deny-s3-access
ListAccessPoints	May 12, 2025, 15:18:57 (UTC-05:00)	chaithrakc	s3.amazonaws.com	-	-
GetBucketWebsite	May 12, 2025, 15:18:57 (UTC-05:00)	chaithrakc	s3.amazonaws.com	-	-
GetBucketPolicy	May 12, 2025, 15:18:57 (UTC-05:00)	chaithrakc	s3.amazonaws.com	-	-
GetAccountPublicAcc...	May 12, 2025, 15:18:50 (UTC-05:00)	chaithrakc	s3.amazonaws.com	-	-
DescribeAvailabilityZ...	May 12, 2025, 15:18:50 (UTC-05:00)	chaithrakc	ec2.amazonaws.com	-	-

◆ Storing Logs Beyond 90 Days

If you want to **retain logs longer than 90 days**:

- Create a **CloudTrail Trail**
- Store logs in a designated **Amazon S3 bucket**
- This enables **long-term audit history** and centralized logging

Permission Boundary



! Note: Permission Boundaries are **rarely used in day-to-day AWS operations**, but are helpful to understand for **interviews** and theoretical purposes. Not a high-priority topic in real-world implementations.

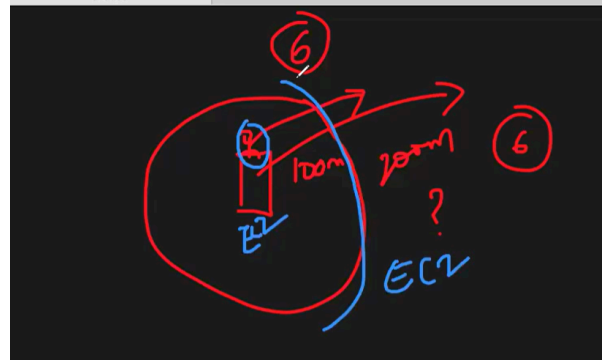
Permissions boundary is to set a maximum permissions for a user can get.

A **Permission Boundary** is a **policy that defines the maximum set of permissions** an IAM user or role can have. Even if other policies grant broader access, the **boundary acts as an**

upper limit —no action beyond it will be allowed.

◆ Example Scenario

- You create an **IAM user** and set a **permission boundary** that allows only **EC2 Full Access**.
- Even if you later attach additional policies like **S3 Full Access**, the user **won't be able to access S3**, because the boundary restricts them to EC2 actions only.



🧠 Think of it as a fence:
permissions inside the fence
(boundary) are possible, but
anything outside is blocked—
regardless of what other policies
say.

◆ Interview Insight: Common Question

Q: A user has a policy granting full access, but still can't access a service. What would you check?

A:

- First, review the IAM policies directly attached to the user.
- If everything seems correct, **check if a permission boundary is applied.**
- The boundary could be **limiting the user's access**, even though broader permissions exist.
- You can also use the **IAM Policy Simulator** to troubleshoot and determine what's blocking access.



Important Notes

- A permission boundary **can contain multiple services**, but you must define them all in a **single policy**.
- You can **only attach one permission boundary** per user or role.
- It **does not grant access by itself** — it **only limits the maximum access possible**.

Testing Permission Boundary

Step 1: Create an IAM user and attach S3 full access policy and also set boundary permission with S3 full access.

Console sign-in URL:

<https://awsprimary-accnt.signin.aws.amazon.com/console>

User name: test-permission-boundary

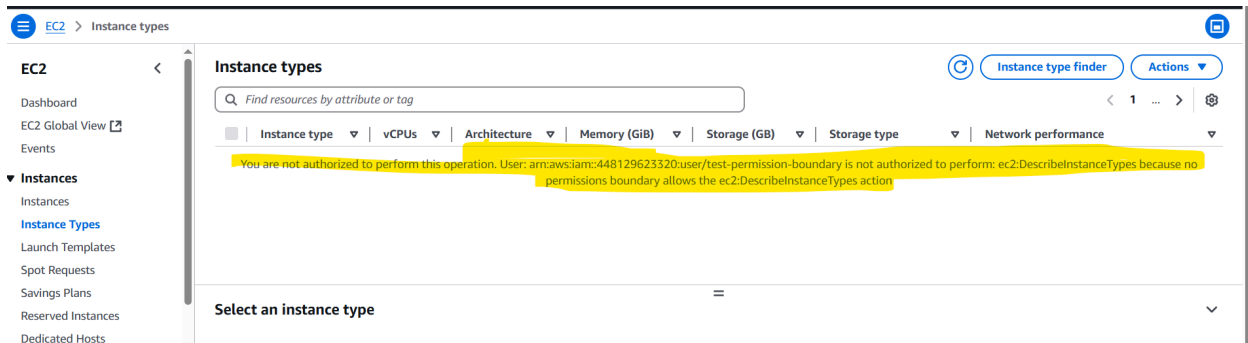
Console password: CustomPassword0*

Add EC2 full access to the permissions:

The screenshot shows the 'Review and create' step in the AWS IAM console. The user name is 'test-permission-boundary'. The console password type is 'Custom password'. The 'Require password reset' option is 'No'. Under 'Permissions summary', two policies are listed: 'AmazonS3FullAccess' (AWS managed) and 'AmazonS3FullAccess' (AWS managed) with a yellow highlight on the word 'boundary'. The 'Tags' section is optional and currently empty. At the bottom, there are 'Cancel', 'Previous', and 'Create user' buttons.

The screenshot shows the 'Permissions' tab in the AWS IAM console for a user. It displays 'Permissions policies (2)' and a table with two rows: 'AmazonEC2FullAccess' and 'AmazonS3FullAccess', both 'AWS managed' and 'Attached via Directly'. Below this, the 'Permissions boundary (set)' section shows 'AmazonS3FullAccess' as the selected boundary. At the bottom, there are 'Change boundary' and 'Remove boundary' buttons.

Error: You are not authorized to perform this operation. User: arn:aws:iam::448129623320:user/test-permission-boundary is not authorized to perform: ec2:DescribeInstanceTypes because no permissions boundary allows the ec2:DescribeInstanceTypes action



AWS IAM users have two types of access:

1. **Management Console Access:** Uses a username, password, and sign-in URL. Users can log in through a web browser to perform required activities.
2. **Programmatic Access:** Uses an “Access Key” and “Secret Key”. Users can configure these credentials in command prompts or terminals to execute queries for working with AWS resources.